Pantalla de inicio

Esta guía arma, desde un proyecto vacío, la pantalla que está hoy en la app: una imagen de fondo, el logo, los botones **Iniciar sesión** y **Registrarse**, y el texto de derechos reservados.

Los botones no llevan color propio. Heredan el color `primary` del tema. Los clics todavía no abren otra pantalla.

1. El proyecto

En Android Studio crea un proyecto **Empty Activity** con Jetpack Compose.

- Name: `MyApplication`
- Package name: `cl.duoc.myapplication`
- Language: Kotlin

Ese proyecto ya trae Compose y Material 3. En `app/build.gradle.kts` deben estar estas dependencias. Si el asistente las dejó, no agregues nada más:

```
implementation(platform(libs.androidx.compose.bom))
implementation(libs.androidx.activity.compose)
implementation(libs.androidx.compose.material3)
implementation(libs.androidx.compose.ui)
implementation(libs.androidx.compose.ui.graphics)
implementation(libs.androidx.compose.ui.tooling.preview)
```

- `compose.ui` dibuja lo que se ve: imágenes, columnas y espacios.
- `material3` trae `Button`, `Text` y `MaterialTheme`. El botón sale con el color del tema sin pedirlo en cada pantalla.
- `activity.compose` permite que `MainActivity` muestre funciones composables.
- El BOM fija versiones compatibles entre esas librerías.

Sincroniza Gradle si Android Studio lo pide.

2. Las imágenes

Copia dos archivos a `app/src/main/res/drawable/`:

- `images`: la foto de fondo. En el código se usa como `R.drawable.images`.
- `logo`: el logo. En el código se usa como `R.drawable.logo`.

El nombre del archivo, sin extensión, es el nombre del recurso. `images.png` se llama `R.drawable.images`. Si el archivo se llama distinto, el proyecto no compila.

3. El color de la marca

En `app/src/main/java/cl/duoc/myapplication/ui/theme/Color.kt`, al final, agrega:

```
val BlueDuoc = Color(0xFF000000L)
```

`Color` guarda un color. `0xFF000000` es negro: `FF` es la opacidad total y `000000` es el color.

`BlueDuoc` es el nombre con el que el tema va a pedir ese color. Hoy el valor es negro, así que los botones se ven negros. Para cambiarlos de color basta con cambiar este número. Las pantallas no se tocan.

4. El tema

En `ui/theme/Theme.kt` el color principal de las dos paletas pasa a ser `BlueDuoc`:

```
private val DarkColorScheme = darkColorScheme(
    primary = BlueDuoc,
    secondary = PurpleGrey80,
    tertiary = Pink80
)

private val LightColorScheme = lightColorScheme(
    primary = BlueDuoc,
    secondary = PurpleGrey40,
    tertiary = Pink40
)
```

`primary` es el color que Material usa en los botones. `LightColorScheme` se usa con el teléfono en modo claro y `DarkColorScheme` en modo oscuro. Las dos apuntan a `BlueDuoc`, así que el botón se ve igual en ambos modos.

En la misma función `MyApplicationTheme`, apaga el color dinámico:

```
dynamicColor: Boolean = false,
```

Con `true`, en Android 12 o superior el teléfono ignora `BlueDuoc` y pinta la app con los colores del fondo de pantalla. Con `false`, la app usa las paletas de arriba.

`MaterialTheme(...)` reparte esa paleta a todo lo que esté dentro. Un `Button` sin color propio lee `primary` desde ahí.

5. La pantalla

El archivo es `ui/HomeScreen.kt`. Se arma de a poco. Después de cada bloque, abre la preview y refresca: así se ve qué agregó ese pedazo.

5.1 La función y la preview

Deja el archivo así:

```
package cl.duoc.myapplication.ui

import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.ui.tooling.preview.Preview
import cl.duoc.myapplication.ui.theme.MyApplicationTheme

@Composable
fun HomeScreen() {
    Text("Inicio")
}

@Preview(showBackground = true)
@Composable
fun HomeScreenPreview() {
    MyApplicationTheme {
        HomeScreen()
    }
}
```

`HomeScreen` es la pantalla. `@Composable` le dice a Compose que esta función dibuja interfaz.

`Text("Inicio")` es un texto provisional. Sirve para comprobar que la función se ve antes de poner fotos y botones.

`@Preview` dibuja `HomeScreen` en el panel de Android Studio, sin correr la app. Va dentro de `MyApplicationTheme` para usar la misma paleta que el teléfono.

En la preview debe aparecer la palabra Inicio.

5.2 El fondo

Agrega estos imports:

```
import androidx.compose.foundation.Image
import androidx.compose.foundation.layout.Box
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.ui.Modifier
import androidx.compose.ui.layout.ContentScale
import androidx.compose.ui.res.painterResource
import cl.duoc.myapplication.R
```

Reemplaza el cuerpo de `HomeScreen` por esto:

```
@Composable
fun HomeScreen() {
    Box {
        Image(
            painterResource(R.drawable.images),
            contentDescription = "",
            modifier = Modifier.fillMaxSize(),
            contentScale = ContentScale.Crop
        )
    }
}
```

`Box` apila lo que pongas dentro. El primer elemento queda atrás y el siguiente, encima.

`Image` carga `R.drawable.images`, la foto del paso 2. `fillMaxSize` la estira al tamaño de la pantalla. `ContentScale.Crop` la recorta para cubrir todo el espacio sin deformarla.

El texto “Inicio” ya no está. En la preview se ve solo la foto.

5.3 La columna encima de la foto

Agrega estos imports:

```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.ui.Alignment
```

Dentro del `Box`, debajo de la `Image`, agrega la columna:

```
@Composable
fun HomeScreen() {
    Box {
        Image(
            painterResource(R.drawable.images),
            contentDescription = "",
            modifier = Modifier.fillMaxSize(),
            contentScale = ContentScale.Crop
        )

        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxSize()
        ) {
            Text("Inicio")
        }
    }
}
```

La `Column` ocupa toda la pantalla y queda encima de la foto, porque está después de la `Image` dentro del mismo `Box`.

`CenterHorizontally` centra cada hijo de izquierda a derecha. `Arrangement.Center` los junta al medio de arriba a abajo.

`Text("Inicio")` vuelve solo para ver la columna sobre la foto. En el paso siguiente lo sacamos.

5.4 El logo

Debajo de `HomeScreen`, agrega esta función:

```
@Composable
fun LogoApp() {
    Image(
        painter = painterResource(R.drawable.logo),
        contentDescription = "Logo App",
        modifier = Modifier.padding(40.dp)
    )
}
```

Y el import del margen:

```
import androidx.compose.foundation.layout.padding
import androidx.compose.ui.unit.dp
```

Dentro de la `Column`, cambia el texto provisional por el logo:

```
Column(
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center,
    modifier = Modifier.fillMaxSize()
) {
    LogoApp()
}
```

`LogoApp` es otra función composable. Muestra `R.drawable.logo`. `padding(40.dp)` deja 40 dp de aire alrededor de la imagen.

Separarla en su propia función deja `HomeScreen` más corta: la pantalla pide el logo, y el detalle de cómo se dibuja vive en `LogoApp`.

En la preview se ve la foto de fondo y el logo al centro.

5.5 Los botones

Agrega este import:

```
import androidx.compose.material3.Button
import androidx.compose.material3.ExperimentalMaterial3Api
```

Encima de `fun HomeScreen` agrega la anotación, y dentro de la columna, debajo de `LogoApp()`, los dos botones:

```
@OptIn(ExperimentalMaterial3Api::class)
@Composable
fun HomeScreen() {
    Box {
        Image(
            painterResource(R.drawable.images),
            contentDescription = "",
            modifier = Modifier.fillMaxSize(),
            contentScale = ContentScale.Crop
        )

        Column(
            horizontalAlignment = Alignment.CenterHorizontally,
            verticalArrangement = Arrangement.Center,
            modifier = Modifier.fillMaxSize()
        ) {
            LogoApp()

            Button(
                onClick = {}
            ) {
                Text("Iniciar Sesión")
            }

            Button(
                onClick = {}
            ) {
                Text("Registrarse")
            }
        }
    }
}
```

`Button` es un componente de Material. No lleva color: lee `primary` del tema, que es `BlueDuoc`. Por eso salen negros.

`onClick = {}` es un clic vacío. El botón se puede pulsar y todavía no abre otra pantalla.

`@OptIn(ExperimentalMaterial3Api::class)` permite usar piezas de Material 3 que siguen marcadas como prueba. En esta pantalla el botón funciona con o sin ella; la dejamos porque el proyecto ya la usa.

En la preview, debajo del logo, aparecen los dos botones.

5.6 El texto de abajo

Agrega este import:

```
import androidx.compose.foundation.layout.Spacer
```

La columna queda así:

```
Column(
    horizontalAlignment = Alignment.CenterHorizontally,
    verticalArrangement = Arrangement.Center,
    modifier = Modifier.fillMaxSize()
) {
    Spacer(modifier = Modifier.weight(1F))

    LogoApp()

    Button(
        onClick = {}
    ) {
        Text("Iniciar Sesión")
    }

    Button(
        onClick = {}
    ) {
        Text("Registrarse")
    }

    Spacer(modifier = Modifier.weight(1F))
    Text(
        "Todos los derechos reservados",
        modifier = Modifier.padding(20.dp)
    )
}
```

`Spacer` es un espacio invisible. `weight(1F)` le da una parte del alto que sobra.

Hay dos espacios con el mismo peso. El primero empuja logo y botones hacia el centro. El segundo ocupa el resto y deja el texto de derechos abajo, con 20 dp de margen.

En la preview ya está la pantalla completa: fondo, logo, dos botones y el aviso inferior.

6. Mostrarla al abrir la app

En `MainActivity.kt`, el `setContent` queda así:

```
setContent {
    MyApplicationTheme {
        HomeScreen()
    }
}
```

`setContent` es lo que la actividad muestra al abrirse. `MyApplicationTheme` aplica la paleta y `HomeScreen()` dibuja la pantalla de inicio.

Import de la pantalla:

```
import cl.duoc.myapplication.ui.HomeScreen
import cl.duoc.myapplication.ui.theme.MyApplicationTheme
```

Sin el import de `HomeScreen`, `MainActivity` no encuentra la función porque está en el paquete `ui`.

Corre la app. Debe verse el fondo, el logo, los dos botones negros y el texto inferior. El negro sale de `BlueDuoc`. Si cambias ese color en `Color.kt` y vuelves a correr, los botones cambian solos.